

# PoPL Tutorial Question Bank

Ramprasad S Joshi

October 2024

**Q1.** Examine the following piece of C code and answer questions below:

```
extern void Print(int*);  
#define Permute(a,b,c) {Exchange(a,b,c); Print(a); Exchange(a,b,c); Print(a);}  
extern void Exchange(int *array, int index1, int index2);
```

- a) Will the code compile successfully?
- b) Will there be a warning about implicit declaration of function Exchange during compilation? Why?
- c) Is the scope of names a,b,c lexical or dynamic? Why?

**Q2.** What will happen if each ASCII character is regarded as a distinct token? Is it possible to build a parser in this case? What problems will you face?

**Q4.** Is the scope of member names in a struct in C **(a)** local to the struct object instance or **(b)** local to the block containing the relevant struct object instance, or **(c)** global?

**Q5.** Can we name members in two different structs in C with the same name? If your answer is yes, explain how a potential name conflict is avoided or resolved. If your answer is no, explain why not.

**Q6,7.** Both the above questions for C++ and/or Java.

**Q8.** To allow call and return by reference, each object created in Java should have a unique entry in the symbol table associated with the running program at runtime. For different objects having the same name, will there be different entries? How are scope rules enforced then?

**Q9.** Exactly what are the types of memory allocation that the C language offers (i.e. a C compiler is supposed to handle)? What is the role of the C compiler in these various memory allocation mechanisms available to a C programmer?

**Q10.** If Java runtime environment allocates each object on the heap (of course—we use the **new** operator—and it is passed around by reference, is there any difference between Java and C/C++ in procedure activation records? Explain in detail and with rationale.

**Q11** What is the type of allocation of arrays defined as follows : “scanf(“%d”,&n); int array[n];”?